

My First Python Game: The Secret Number Challenge!

Learn how to speak to computers and build your own guessing game.



Write real Python code lines and win digital ladoos!

Hi Super Coder! Welcome to our coding lab. Today, you are going to use a special computer language called **Python**. You will build a fun game where the computer hides a secret number and challenges your friends to find it!

Welcome to Our Coding Adventure!

Before we start writing code, let us look at the amazing superpowers you are going to learn today:

Our Fun Coding Goals:

- ★ **Sweet Boxes (Variables):** Learn how the computer remembers information.
- ★ **Listening Blocks (Inputs):** Teach your game to listen to what players type.
- ★ **Umpire Logic (If/Else):** Tell your game how to decide if a guess is right or wrong.
- ★ **The Alignment Trick (Indentation):** Learn how to keep your code lines tidy.

By the end of this workbook, you will be a real game developer with a game that you built all by yourself!

Concept 1: What is a Variable?

Imagine your kitchen counter has shiny, silver **steel containers** lined up. Each container has a neat sticker label on the front so you know what is hiding inside.

Ladoos and Barfis!

One box is labeled "**Ladoos**" and has 5 sweet ladoos inside. Another box is labeled "**Barfis**". In coding, these labeled steel containers are called **Variables**!

How Computers Use Containers

A variable is just a storage box with a name. We can change what is inside the box whenever we want! In our game, we will make a storage box named **secret_number** to hold our hidden answer.

```
secret_number = 5
```

 This line tells Python: "Take a container, label it 'secret_number', and drop the number 5 inside!"

Concept 2: Taking Player Input!

How does the computer listen to what you want? Imagine you walk up to your favorite street food stall to order yummy, crunchy **Golgappas**.



The Golgappa Vendor Test

The vendor stands behind the counter and asks you: *"What will you have? How many golgappas do you want?"* You yell out your order, and they listen carefully.

The input() Question Mark

In Python, we use a special tool called **input()**. It works exactly like that friendly vendor! It stops the game, prints a question on the screen, and listens closely for the player to type their guess.

```
guess = input("Guess a number: ")
```

This tells the computer to listen to the player's guess and catch it inside a container named **guess**!

Concept 3: If / Else Decisions

Computers need to make big decisions based on rules. To understand how they do this, let's step onto a cricket ground!

The Gully Cricket Umpire Rule:

***"IF** the ball hits the wickets → The batsman is OUT!*

***ELSE** → The batsman is NOT OUT!"*

Teaching Python to be the Umpire

Our guessing game needs to make an umpire decision too! We use **if** and **else** blocks to check the player's answer:

```
if guess == secret_number:  
    print("Correct! You win!")  
else:  
    print("Try again!")
```

The Metro Train Rule (Indentation)

Before we put our lines together, we must learn a critical spacing rule called **Indentation**. It's all about keeping lines neatly aligned!



Standing Inside the Metro Car

Imagine you are boarding a fast Metro train. To ride safely, your whole body has to step completely **INSIDE** the train doors. You can't leave one foot outside on the platform!

Stepping Inside the Code Block

In Python, when you write code that belongs inside an **if** or **else** rule, you must push those lines forward by pressing the **Tab key** on your keyboard. This pushes the line "inside the train doors" so Python knows exactly which rule it belongs to!

Our Complete 5-Line Game Script!

Let's assemble our code blocks like a puzzle! Here is your complete 5-line game program. Look at how beautifully aligned it is:

```
# Line 1: Store our hidden treat!
secret_number = 5

# Line 2: Ask for the player's choice!
guess = int(input("Guess a number: "))

# Line 3-4: The Umpire checks if it is correct!
if guess == secret_number:
    print("Correct! You win a ladoo! 🥰")

# Line 5-6: Oops, tell them to try again!
else:
    print("Wrong guess! Try again! 😞")
```

💡 Note: We wrapped our input inside an **int()** block to turn the player's typed letters into a real math number!

Bug Hunting: Help Fix the Code Errors!

Sometimes, an annoying little insect crawls into our code and makes it stop working. Coders call these mistakes **Bugs**. Let's learn to hunt them down!

3 Common Bug Traps to Avoid:

- **The Missing Wall (Indentation Error):** Forgetting to press the Tab key to push your print line inside the if/else doors!
- **The Forgotten Two-Dots (Colon `:`):** Forgetting to put the double dots symbol at the very end of your if and else lines.
- **Funny Word Spellings:** Typing `print ( )` instead of `print ( )`. Python only understands perfect word spells!

If your code shows a red error message, do not worry! Just check these 3 traps, fix them up, and run it again.

Make Your Guessing Game Unique!

Hooray! You know the core engine lines. Now you can act like a true game designer and add your own fun rules to the script!

Awesome Ideas to Try Out:

- ★ **Change the Hiding Spot:** Switch the secret number from 5 to your lucky age, like 7 or 8!
- ★ **Custom Victory Screams:** Change the win message to say *"Boom! You are a master champion! 🏆"*
- ★ **Add Score Rewards:** Give players imaginary gold coins or bonus points for matching your number on the first try.

Experiment with your text lines and see how the computer displays your personal changes instantly!

Congratulations, Master Python Coder!

You have successfully unlocked the magical secrets of computer science basics today! Let's review what you achieved:

☀️ Your Coding Certificate Checklist: ☀️

- ✓ Created storage variables like sweet containers.
- ✓ Used input() to listen to player commands.
- ✓ Code with clean, safe Metro-style Indentation spacing.



Certified Junior Python Developer!



**Now open up your code editor, type out your 5 script lines,
hit the RUN button, and invite your friends to play!**